Resumen de la sesión 2

TPC-DS en Parquet sobre HDFS



Curso 2026/2027

Diego Sevilla
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

Qué vamos a repasar

- **Data lake y arquitectura medallion**: capas `raw`, `silver` y `gold`
- **El dataset**: TPC-DS SF3 generado con DuckDB y adaptado al curso
- **Los municipios de España**: el CSV del IGN y el `JOIN` determinista
- **La capa `raw` en HDFS**: directorios, marcadores `_SUCCESS` y tamaños
- **DuckDB y las vistas temporales**: consultar Parquet como si fueran tablas
- **Las tablas principales**: hechos, dimensiones y claves
- **Versiones con doble fecha**: qué fila estaba vigente cada día
- **Cuatro consultas** que volveremos a ver en S3

Data lake y medallion

Dónde se guardan los datos y cómo se ordenan

Data lake frente a data warehouse

- **Warehouse**: el esquema se define antes de cargar (*schema-on-write*); sólo entra lo que encaja en las tablas diseñadas
- **Data lake**: los ficheros se guardan como llegan, en un almacenamiento distribuido (HDFS, S3…), y el esquema se interpreta al leer (*schema-on-read*)
  - Se conserva el dato original y se puede **reprocesar** todo el histórico
  - Se incorporan fuentes distintas sin negociar antes un esquema común
- El riesgo: un ***data swamp***, donde nadie sabe qué fichero es fiable

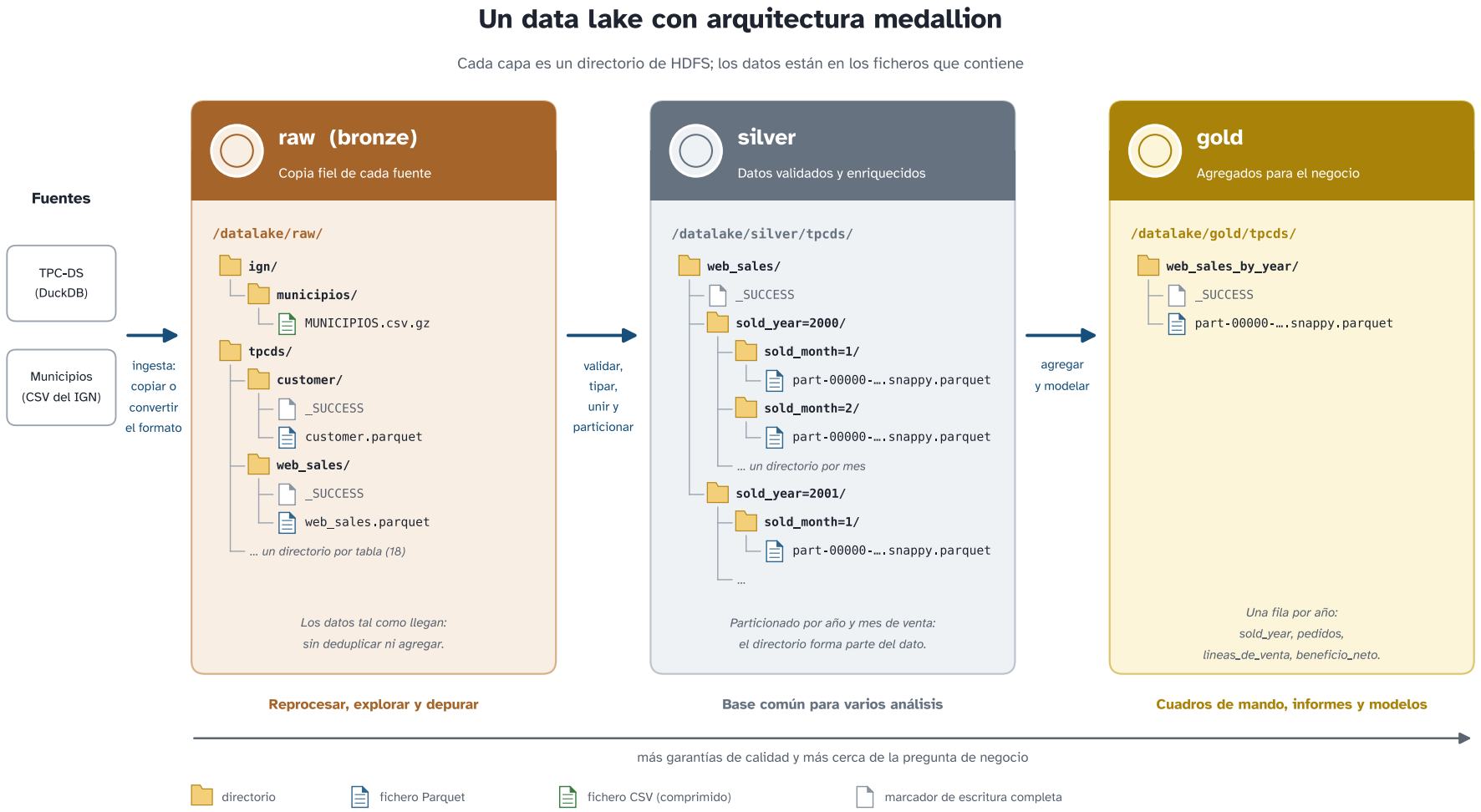
> La arquitectura **medallion** ordena el lake en capas, cada una con un contrato sobre lo que contiene

Las tres capas

Capa	Qué guarda	Cómo se llega	Para qué sirve
`raw` (*bronze*)	La fuente tal como llega	Copia o conversión de formato	Reprocesar sin volver a la fuente
`silver`	Datos validados, tipados, sin duplicados, unidos entre fuentes	Limpieza y `JOIN`	Base común para muchos consumidores
`gold`	Agregados para una pregunta de negocio	Agregación sobre `silver`	Cuadros de mando, informes, modelos

- En el curso, `raw` es **el punto de partida**: el CSV del IGN tal cual y las tablas TPC-DS ya adaptadas, que tomamos como datos originales
- S2 escribe `raw`; `silver` y `gold` llegarán en la sesión 5
- No es una jerarquía obligatoria, sino un **compromiso de calidad**: se puede leer `raw` para explorar

El data lake del curso



El dataset

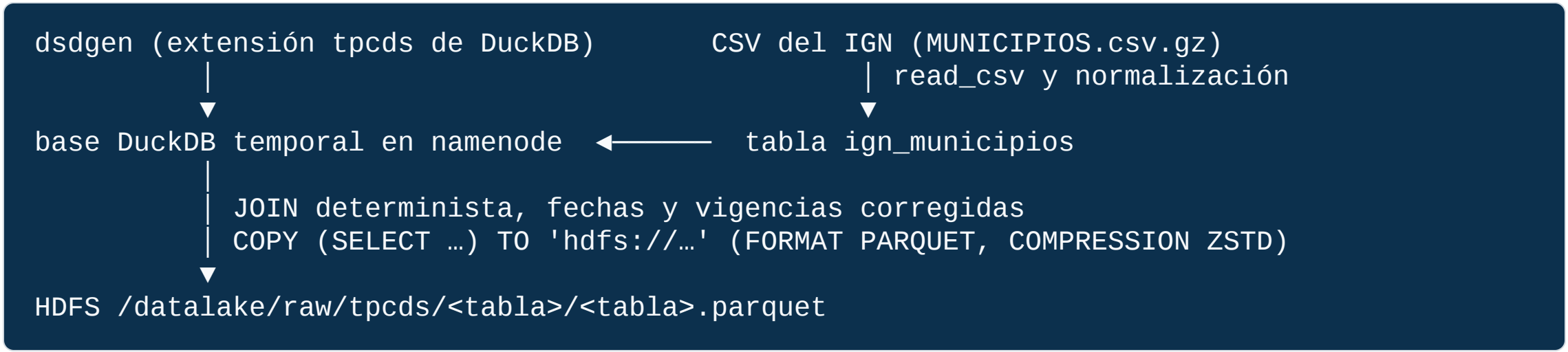
TPC-DS SF3, generado y adaptado con DuckDB

TPC-DS SF3

- Benchmark de apoyo a la decisión: una cadena **minorista** que vende en tienda, por catálogo y por web
- **Dimensiones** (clientes, productos, tiendas…) y **hechos** (ventas, devoluciones, inventario), unidos por claves `_sk`
- El curso usa **18 tablas**
- **SF3** ≈ 3 GB sin comprimir → unos **545 MiB en Parquet** (1,6 GiB con las tres réplicas de HDFS) y unos **42 millones de filas**
- Determinista: con la misma versión de DuckDB, todo el mundo obtiene las mismas filas

> El generador `dsdgen` viene como **extensión** de DuckDB: `INSTALL tpcds; LOAD tpcds; CALL dsdgen(sf = 3)`

La ruta de los datos



- DuckDB es una base de datos **embebida**: una biblioteca dentro del proceso de Python, sin servidor
- La base temporal se **borra** al terminar: lo que queda está en HDFS
- DuckDB escribe en HDFS mediante `hdfs-native`, un cliente de HDFS sin Java

Los municipios de España

Una segunda fuente, real, cruzada con la sintética

El Nomenclátor del IGN

8.132 municipios con provincia, población, superficie, coordenadas y altitud (NGMEP CC-BY 4.0 ign.es)

```
SELECT * FROM read_csv('MUNICIPIOS.csv.gz',
    delim = ';',                  -- la coma es el separador decimal
    encoding = 'latin-1',         -- no es UTF-8: «Araba/�lava»
    decimal_separator = ',',      -- 1994,5872
    types = {'COD_INE': 'VARCHAR', 'COD_PROV': 'VARCHAR'})  -- conservar los ceros iniciales
```

- Antes de leer un CSV público hay que **mirar sus bytes**: separador, codificación, decimales y códigos
- Se valida: 8.132 municipios únicos, provincias `01` – `52`, coordenadas dentro de España
- El CSV se guarda **tal cual**, en `/datalake/raw/ign/municipios`

Un `JOIN` determinista

Cada pareja ciudad–estado de TPC-DS elige siempre el mismo municipio:

```
JOIN ign_municipios_ordenados AS ign
  ON ign.posicion = 1 + md5_number(
      concat(customer_address.ca_city, '|', customer_address.ca_state)
  ) % 8132
```

- `posicion` numera los municipios del 1 al 8.132, sin huecos
- **No se usa `random()`**: repetir la generación da el mismo resultado en cualquier equipo
- `concat` trata `NULL` como cadena vacía; con `||` la fila perdería el `JOIN`
- Se sustituyen `*_city` → municipio, `*_county` → provincia, `*_state` → código de provincia y `*_country` → `España`
- Calles, códigos postales y husos horarios **siguen siendo sintéticos**: el IGN no los aporta

La capa `raw` en HDFS

Qué ha quedado escrito y cómo comprobarlo

Un directorio por tabla

```
/datalake/raw/
├── ign/municipios/MUNICIPIOS.csv.gz
└── tpcds/
    ├── customer/       customer.parquet          _SUCCESS  (vacío)
    ├── customer_address/ customer_address.parquet  _SUCCESS  INE-2026
    ├── item/           item.parquet              _SUCCESS  SCD2
    ├── store/          store.parquet             _SUCCESS  INE-2026+SCD2+FECHAS
    ├── store_sales/    store_sales.parquet       _SUCCESS  SCD2+FECHAS
    └── …               (18 tablas)
```

- Una **tabla es un directorio**, no un fichero: Spark escribirá varios ficheros en el mismo directorio
- El formato se reconoce por la firma `PAR1` del fichero, no por la extensión
- `hdfs dfs -du -h` da dos tamaños: el **lógico** y el **ocupado con réplicas** (×3)

DuckDB y las vistas temporales

De ficheros a tablas con nombre

Una vista por tabla

```
duckdb.register_filesystem(ThreadSafeHdfs(host="namenode", port=9000),
                           connection=connection)

def create_view(table: str) -> None:
    url_list = ", ".join(f"'{url}'" for url in table_urls(table))   # sin _SUCCESS
    connection.execute(
        f"CREATE OR REPLACE VIEW {table} AS SELECT * FROM read_parquet([{url_list}])")

create_view("web_sales")
connection.sql("SELECT count(*) FROM web_sales")       # 2.160.165 filas
```

- `register_filesystem` le da a DuckDB el sistema de ficheros de `fsspec` para leer URL `hdfs://`
- La lista excluye `_SUCCESS`: no es un Parquet y DuckDB fallaría al leerlo

Qué es una vista (y qué no)

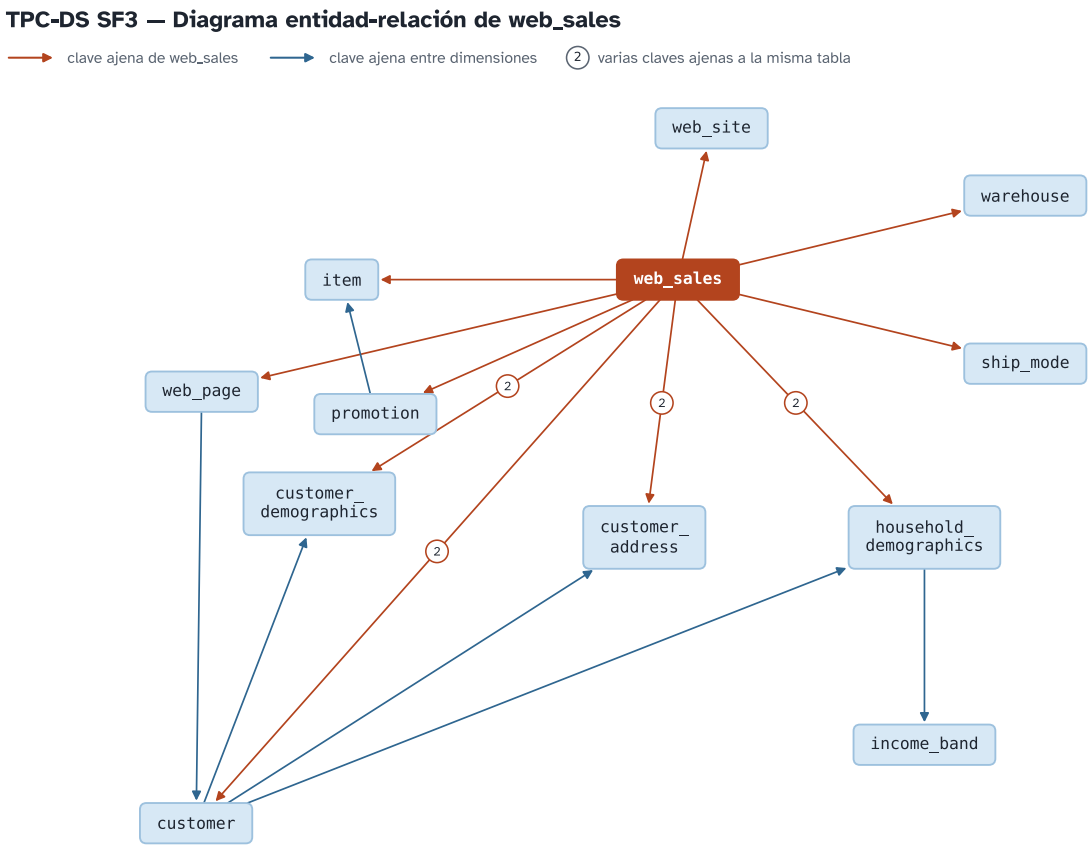
- Una vista guarda la **consulta**, no los datos: no copia nada
- Cada consulta sobre `web_sales` **vuelve a leer** los Parquet de HDFS (aunque DuckDB es muy eficiente con las cachés)
- Desaparece al cerrar la conexión: es temporal
- Las consultas se escriben como con tablas normales: `FROM web_sales`
- PyArrow, Polars y DuckDB leyeron `item` y coincidieron en filas, no nulos, mínimo, máximo y suma de `i_item_sk`

> En **S3** sólo cambian las URL (`s3://tcdm-datalake/raw/tpcds/…`) y DuckDB las lee con `httpfs`.
> En **S6** la tabla tendrá un nombre en el catálogo: `tcdm.web_sales`

Las tablas principales

Hechos, dimensiones y claves

Un hecho rodeado de dimensiones



- El **prefijo** dice de qué tabla es una columna: `ws_` es `web_sales`, `c_` es `customer`…
- Las columnas `_sk` son **claves**: la propia o la que apunta a otra tabla
- Un hecho son casi todo claves, más cantidades e importes
- Las claves ajenas son **lógicas**: Parquet no las impone

Las tablas que más usaremos

Tabla	Una fila es…	Columnas para recordar
`web_sales`	una línea de un pedido web	`ws_order_number`, `ws_item_sk`, `ws_bill_customer_sk`, `ws_bill_addr_sk`, `ws_sold_timestamp`
`store_sales`	una línea de un ticket de tienda	`ss_ticket_number`, `ss_item_sk`, `ss_customer_sk`, `ss_store_sk`, `ss_sold_timestamp`
`item`	una **versión** de un producto	`i_item_sk`, `i_item_id`, `i_product_name`, `i_category`
`customer`	un cliente	`c_customer_sk`, `c_first_name`, `c_last_name`, `c_current_addr_sk`
`customer_address`	un domicilio	`ca_address_sk`, `ca_city` (municipio), `ca_county` (provincia), `ca_state` (código)

- **Una fila de un hecho es una línea, no un pedido**: `count(*)` cuenta líneas y `count(DISTINCT ws_order_number)` cuenta pedidos
- Las medidas tienen el mismo nombre en todas las sesiones: `lineas_de_venta`, `pedidos`, `tickets`, `unidades`, `ventas`, `beneficio_neto`

Una venta, clave a clave

```sql
SELECT web_sales.ws_order_number,
       item.i_product_name AS producto,
       customer.c_first_name || ' ' || customer.c_last_name AS cliente,
       customer_address.ca_city AS municipio,
       CAST(web_sales.ws_sold_timestamp AS DATE) AS fecha
FROM web_sales
JOIN item             ON web_sales.ws_item_sk = item.i_item_sk
JOIN customer         ON web_sales.ws_bill_customer_sk = customer.c_customer_sk
JOIN customer_address ON web_sales.ws_bill_addr_sk = customer_address.ca_address_sk
WHERE web_sales.ws_order_number = 1
```

- El patrón de toda consulta analítica: **un hecho y un `JOIN` por dimensión**
- Hay dos caminos a un lugar: el domicilio de facturación del pedido (`ws_bill_addr_sk`) y el actual del cliente (`c_current_addr_sk`). No coinciden: hay que decir cuál se usa

Versiones con doble fecha

Dimensiones que guardan su historia

Una fila por versión (SCD 2)

- `item`, `store`, `web_page` y `web_site` guardan **una fila por versión** de cada objeto: si cambia el precio, se añade una fila y la anterior se cierra
  - `_sk`: clave de **la versión**; es la que guardan las ventas
  - `_id`: clave de negocio, **la misma** en todas las versiones
  - `rec_start_date` y `rec_end_date`: primer y último día de vigencia; `rec_end_date` es `NULL` en la **versión vigente**
- Los periodos de un objeto se suceden **sin solaparse**
- Una venta queda enlazada al producto tal como era **el día en que se hizo**

> Es una *slowly changing dimension* de tipo 2 (SCD 2)

Un objeto, varias versiones: las dimensiones SCD 2 de TPC-DS

Cada versión es una fila con su propia _sk; las versiones del mismo objeto comparten _id. Primeras seis filas, con las fechas ya corregidas en S2

item — productos · una fila por versión

_sk · clave primaria
Una distinta por versión. La guardan las ventas.

_id · clave de negocio
La misma en todas las versiones del mismo objeto.

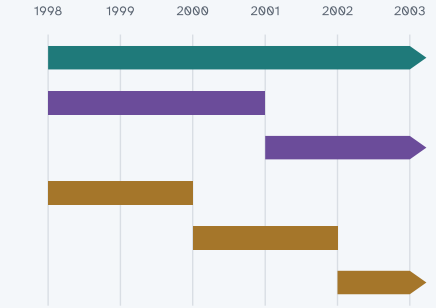
rec_start_date · rec_end_date
Primer y último día de la versión. rec_end_date vacío (NULL): versión vigente.

lo que cambia
Otra versión, otros datos.

i_item_sk	i_item_id	i_rec_start_date	i_rec_end_date	i_current_price
1	AAAAAAAABAAAAAAA	1998-01-01	NULL	27.02
2	AAAAAAAACAAAAAAA	1998-01-01	2000-12-31	1.12
3	AAAAAAAACAAAAAAA	2001-01-01	NULL	7.11
4	AAAAAAAAEAAAAAAA	1998-01-01	2000-01-01	1.35
5	AAAAAAAAEAAAAAAA	2000-01-02	2001-12-31	4.00
6	AAAAAAAAEAAAAAAA	2002-01-01	NULL	0.85

1 versión — 1 _sk · 1 _id
2 versiones — 2 _sk · 1 _id
3 versiones — 3 _sk · 1 _id

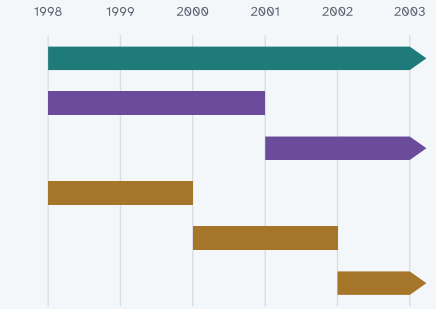
periodos de vigencia
Los de un mismo objeto se suceden sin solaparse. La flecha marca la versión vigente.



store — tiendas · una fila por versión

s_store_sk	s_store_id	s_rec_start_date	s_rec_end_date	s_floor_space
1	AAAAAAAABAAAAAAA	1998-01-01	NULL	5250760
2	AAAAAAAACAAAAAAA	1998-01-01	2000-12-31	5285950
3	AAAAAAAACAAAAAAA	2001-01-01	NULL	7557959
4	AAAAAAAAEAAAAAAA	1998-01-01	2000-01-01	9341467
5	AAAAAAAAEAAAAAAA	2000-01-02	2001-12-31	9078805
6	AAAAAAAAEAAAAAAA	2002-01-01	NULL	9026222

1 versión — 1 _sk · 1 _id
2 versiones — 2 _sk · 1 _id
3 versiones — 3 _sk · 1 _id



La venta guarda la _sk	ws_item_sk o ss_store_sk apuntan a una versión concreta: la vigente el día de la venta.
Contar objetos	`count(DISTINCT i_item_id)`
Sólo la versión vigente	`WHERE i_rec_end_date IS NULL`

Consultar con la doble fecha

```sql
-- Sólo la versión vigente de cada producto
WHERE i_rec_end_date IS NULL

-- La versión que estaba vigente un día concreto
WHERE DATE '2000-06-15' BETWEEN i_rec_start_date
                            AND coalesce(i_rec_end_date, DATE '9999-12-31')
```

- Contar filas de `item` **no es contar productos**: `count(DISTINCT i_item_id)`
- Agrupar por `i_item_sk` separa las versiones; por `i_item_id` las reúne
- El generador fechaba mal las versiones (entre 66 y 294 días de adelanto) y `store_sales` apuntaba siempre a la primera versión de la tienda. S2 lo corrige al escribir
- Comprobación final: **0 líneas de venta** apuntan a una versión fuera de su vigencia

Cuatro consultas

Las mismas que repetiremos en S3 sobre el almacén de objetos

1. ¿Qué cliente ha hecho más pedidos?

```
SELECT customer.c_customer_id,
       concat_ws(' ', customer.c_first_name, customer.c_last_name) AS cliente,
       count(DISTINCT web_sales.ws_order_number) AS pedidos,
       sum(web_sales.ws_ext_sales_price) AS ventas
FROM web_sales
JOIN customer ON web_sales.ws_bill_customer_sk = customer.c_customer_sk
GROUP BY customer.c_customer_id, customer.c_first_name, customer.c_last_name
ORDER BY pedidos DESC, customer.c_customer_id
LIMIT 10
```

- `count(DISTINCT ws_order_number)` cuenta **pedidos**; con `count(*)` serían líneas y el orden cambia
- `concat_ws` no devuelve `NULL` si falta el nombre o el apellido, a diferencia de `||`
- El `ORDER BY` desempata por la clave: el resultado es repetible
- ¿El cliente con más pedidos es el que más ha gastado?

2. ¿De qué provincia salen más pedidos?

```sql
SELECT customer_address.ca_state AS provincia_id,
       customer_address.ca_county AS provincia,
       count(DISTINCT web_sales.ws_order_number) AS pedidos,
       sum(web_sales.ws_ext_sales_price) AS ventas
FROM web_sales
JOIN customer_address
    ON web_sales.ws_bill_addr_sk = customer_address.ca_address_sk
GROUP BY customer_address.ca_state, customer_address.ca_county
ORDER BY pedidos DESC, provincia_id
LIMIT 10
```

- Aquí se ve el cruce con el IGN: `ca_county` y `ca_state` son provincias españolas
- El `JOIN` va por la **clave** de dirección, no por nombres: los nombres sólo describen el resultado
- Las ventas son sintéticas y el reparto de municipios no sigue la población real

3. ¿Cómo se reparten las ventas por mes?

```
SELECT month(ss_sold_timestamp) AS mes,
       count(DISTINCT ss_ticket_number) AS tickets,
       sum(ss_quantity) AS unidades,
       sum(ss_ext_sales_price) AS ventas
FROM store_sales
WHERE year(ss_sold_timestamp) BETWEEN 1998 AND 2002
GROUP BY mes
ORDER BY mes
```

- `month()` sale de la marca de tiempo: no hace falta `JOIN` con un calendario
- En tienda se cuentan **tickets** distintos, el equivalente al pedido web
- El filtro deja sólo años completos: 2003 sólo tiene sus primeros días y se sumaría a enero
- ¿Se reparten las ventas por igual a lo largo del año?

4. ¿Qué producto se vende más cada mes?

```
WITH unidades_por_mes AS (
    SELECT month(store_sales.ss_sold_timestamp) AS mes, item.i_item_sk,
           item.i_product_name AS producto, sum(store_sales.ss_quantity) AS unidades
    FROM store_sales
    JOIN item ON store_sales.ss_item_sk = item.i_item_sk
    WHERE year(store_sales.ss_sold_timestamp) BETWEEN 1998 AND 2002
    GROUP BY mes, item.i_item_sk, item.i_product_name
),
clasificados AS (
    SELECT *, row_number() OVER (PARTITION BY mes
                                 ORDER BY unidades DESC, i_item_sk) AS posición
    FROM unidades_por_mes
)
SELECT mes, producto, unidades FROM clasificados WHERE posición = 1 ORDER BY mes
```

«El más vendido de cada grupo»

- Un `GROUP BY` con `LIMIT` no basta: el `LIMIT` corta el resultado entero, no cada mes
- Se resuelve en tres pasos, uno por bloque:
  i. **Agregar**: unidades de cada producto en cada mes
  ii. **Numerar** dentro de cada mes: `row_number() OVER (PARTITION BY mes ORDER BY unidades DESC)`
  iii. **Filtrar** la posición 1 en una consulta exterior
- El filtro va fuera porque `WHERE` se evalúa **antes** que las funciones de ventana (DuckDB también admite `QUALIFY`)
- Cambiando la partición, la misma consulta responde «el más vendido de cada provincia»

Lo que hay que llevarse de S2

- `/datalake/raw` es el punto de partida: 18 tablas TPC-DS adaptadas y el CSV del IGN
- Una tabla es un **directorio de Parquet**; `_SUCCESS` dice que está completa y con qué correcciones
- DuckDB consulta esos ficheros con SQL dentro de Python; una **vista** les da nombre sin copiar datos
- Los hechos son líneas: cuidado con `count(*)`
- Las dimensiones con versiones se enlazan por `_sk` y se agrupan por `_id`
- Todo ocurre **en un solo proceso** dentro de `namenode`: aún no hay cálculo distribuido

> **En S3** copiamos `raw/tpcds` a un bucket S3 (RustFS) y volvemos a leerlo con PyArrow, Polars y DuckDB: el almacenamiento cambia, las consultas no